

Correction to experiment 01's lead-time figures

Issued 3 October 2026, after re-running the experiment. This page is bound in front of the report it corrects.

What is wrong

Three passages in this report state that on generated data a tuned LSTM beat the world model 3.35 windows to 0.26 at a false-alarm rate at or below 10%. The second of those numbers is stale. It predates two model fixes recorded in RESULTS.md - the ATT&CK head being trained on encoded rather than predicted latents, and training at one step while inferring at six - and the write-ups were never refreshed.

What the experiment produces now

World model: 2.70 windows at 70% detection

LSTM: 3.30 windows at 74% detection

GBDT: 3.54 windows at 63% detection

Source: backend/results/exp01_leadtime.json, reproduced in four consecutive runs.

What does not change

The refutation stands. Prediction Q2 - that the world model would match or beat the LSTM on lead time - still fails, and the headline claim of this project remains refuted. The error ran against us: the world model was less bad than we published.

A second fault, found while checking the first

This experiment's result depends on how many threads the machine gives it. The fused CPU kernel behind nn.LSTM and XGBoost reduce floating-point values in thread order, so a machine with a different core count gets a different answer:

torch's default (6 threads here): LSTM 4.71 windows at 93%, and Q1 holds

pinned to one thread: LSTM 3.30 windows at 74%, and Q1 fails

The experiment now pins itself to one thread, so what it prints is a property of the code rather than of the machine. Two runs agree exactly in either configuration: it is stable run to run, and was never otherwise. Prediction Q1 should be read as thread-dependent rather than settled. Q2, the claim this project is named for, fails under every configuration tested.

Why the gap cannot be attributed

The published figures match neither configuration above, and no results payload records the library versions or the thread count used. A torch upgrade and a different machine are both consistent with the difference, and nothing in the artefacts can separate them - the same lesson experiment 14 taught when it failed to record its epoch count: a knob you do not record is a number you cannot regenerate.

RESULTS.md, README.md, ARCHITECTURE.md and JUDGE_QA.md carry the corrected figures and the full note. This PDF's body is unchanged: its generator no longer exists.

KALACHAKRA

AI-Based Network Attack Forecasting from Network Traffic Data

Problem Statement ID	SIH26153
Problem Statement	AI based Network Attack Forecasting from Network Traffic Data
Organisation	National Technical Research Organisation (NTRO)
Theme	Blockchain and Cybersecurity
PS Category	Software
Team	AlgoRhythms
Document type	Complete technical, functional and presentation reference
Status of figures	All measurements read from the repository's own results files at authoring time
Companion project	EKAGRA (SIH26145) - supplies roughly 60-70% of this project's data foundation

This document is written to be self-sufficient. It is also written to be uncomfortable in places: KALACHAKRA's central capability claim was refuted by the team's own experiments, on generated data and again on real botnet captures. Those refutations are documented here in full, because they are the reason the surviving claims can be trusted.

Table of Contents

1. Executive Summary	6
1.1 What the project is	6
1.2 The problem it solves	6
1.3 Who experiences this problem	6
1.4 How the solution addresses it - and what happened when it was tested	6
1.5 What is genuinely valuable here	6
2. Problem Statement	7
2.1 What the client asked for	7
2.2 The distinguishing question	7
2.3 Classifier versus world model	7
2.4 Why existing approaches are insufficient	7
2.5 Real-world consequences and the specific gap	8
3. Motivation and Relevance	8
3.1 Why this project was created	8
3.2 Why the problem is relevant now	8
3.3 Potential real-world applications	8
4. Proposed Solution	8
4.1 Core idea	8
4.2 Why a graph rather than a flat vector	9
4.3 Why predict in latent space rather than reconstruct traffic	9
4.4 Conceptual workflow	9
5. System Architecture	9
5.1 State representation	9
5.2 Model components	10
5.3 The ATT&CK stage ladder	10
5.4 Training objective	10
5.5 What is deliberately not built	11
6. End-to-End Workflow	11
7. Features	12
7.1 Core features	12
7.2 Data and evaluation features	12
8. Technical Implementation	12
8.1 Notable implementation details	13
9. AI, ML and Data Science Components	13
9.1 Problem formulation	13

9.2 Datasets	13
9.3 The CTU-13 adapter and the confounds it had to remove	14
9.4 Model architecture in detail	14
9.5 Training methodology	14
9.6 Evaluation protocol	14
9.7 Explainability - what is shipped and what is substituted	15
10. Data Stores and Data Flow	15
10.1 Data validation	16
11. APIs and Integrations	16
11.1 The one external data dependency	16
11.2 Internal interfaces that behave like contracts	16
12. User Journey	16
13. User Interface and Experience	17
14. Security and Privacy	17
15. Performance and Scalability	18
15.1 Scalability considerations	18
16. Deployment	18
16.1 Environment	18
16.2 Running it	18
16.3 Production posture	18
17. Project and Codebase Structure	19
17.1 Experiment inventory	19
18. Algorithms and Core Logic	20
18.1 The JEPA-style latent predictive objective	20
18.2 Multi-step rollout with prediction feedback	20
18.3 Compromise-risk aggregation	20
18.4 Lead time at a matched false-alarm rate	20
18.5 Integrated gradients with adaptive refinement	20
18.6 Nested selection - the protocol that prevents self-deception	21
19. Innovation and Differentiation	21
20. Existing Solutions Compared With KALACHAKRA	21
21. Use Cases	22
21.1 Primary - intervention decision support	22
21.2 Secondary use cases	22
22. Advantages	22
23. Limitations and Current Gaps	23

23.1 Capability limitations	23
23.2 What could not be tested	23
23.3 Engineering gaps	23
23.4 Documentation gaps	23
24. Future Scope	24
24.1 Short term	24
24.2 Medium term	24
24.3 Long term	24
25. Testing and Validation	24
25.1 Validation methodology	25
26. Results and Evidence	25
26.1 Generated-data results (experiments 01 and 02)	25
26.2 Real-data results on CTU-13 (experiment 14)	25
26.3 The horizon sweep (experiment 16)	26
26.4 The correction: our own published number was wrong	26
26.5 Measured versus projected	27
26.6 Contradictions found between sources, and the ruling on each	27
27. Risks and Challenges	28
28. Presentation Knowledge Base	29
28.1 Slide 1: Title	29
28.2 Slide 2: Problem	29
28.3 Slide 3: Motivation	29
28.4 Slide 4: Existing System	30
28.5 Slide 5: Proposed Solution	30
28.6 Slide 6: Innovation	30
28.7 Slide 7: Architecture	30
28.8 Slide 8: Workflow	31
28.9 Slide 9: Technologies	31
28.10 Slide 10: AI / ML	31
28.11 Slide 11: Features	31
28.12 Slide 12: Demo Flow	32
28.13 Slide 13: Results	32
28.14 Slide 14: Advantages	32
28.15 Slide 15: Future Scope	32
28.16 Slide 16: Conclusion	33
29. Pitch Material	34
29.1 30-second pitch	34

29.2 1-minute pitch	34
29.3 3-minute pitch	34
29.4 5-minute technical explanation	34
29.5 Explaining it to a judge	34
29.6 Explaining it to a non-technical audience	34
29.7 Technical viva	35
30. Expected Questions and Answers	36
31. Demonstration Script	38
32. Glossary	39
33. One-Page Cheat Sheet	42

1. Executive Summary

1.1 What the project is

KALACHAKRA is a **world model for network state**. Rather than classifying traffic as malicious or benign, it learns the transition dynamics of a host-interaction graph - written $P(S_{t+1} | S_t, \text{action})$ - and rolls those dynamics forward K steps without new observations. That makes two things possible that a classifier cannot do: forecasting whether a trajectory is heading towards compromise, and answering a counterfactual - *what would happen if we isolated this host now?*

1.2 The problem it solves

A security operations centre does not primarily need to know whether something is happening; by the time a classifier fires, the analyst usually knows. What the analyst needs is **how much time is left** and **whether acting now would help**. Detection scores answer neither question. A discriminative model can only score the trajectory it actually observed; it has no mechanism to condition on an action that was never taken.

1.3 Who experiences this problem

NTRO, NCIIPC-designated Critical Information Infrastructure operators, and CERT-In empanelled SOCs. CERT-In handled 29.44 lakh cyber incidents in 2025, up 85% since 2023. At that volume the bottleneck is not detection but triage and decision.

1.4 How the solution addresses it - and what happened when it was tested

The honest position, stated up front

The lead-time claim - that a world model warns earlier than a classifier - is **refuted**. It was refuted on the project's own generated data (a tuned LSTM won 3.35 windows to 0.26) and the refutation replicated on CTU-13, thirteen real botnet captures totalling 19.9 million flows, where the world model catches 5 of 12 real infections and a plain gradient-boosted tree catches 7 to 9. The surviving differentiator is the counterfactual, and even that is reported with a caveat that a trivial baseline beats the headline number.

1.5 What is genuinely valuable here

- **A capability a classifier structurally cannot have.** The transition model takes an action as input. Roll out under 'do nothing' and under 'isolate', and the difference is the model's belief about the intervention. There is no bolt-on that gives a discriminative model this.
- **An evaluation instrument the field does not have.** Lead time at a matched false-alarm rate, leave-one-scenario-out across thirteen real captures, and contained attacks counted as negatives so the task cannot collapse into detection.
- **A finding about the problem, not the team.** On CTU-13 all four methods span roughly 0.03 to 0.99 AUC across captures. Cross-network transfer on this corpus is close to a coin flip per capture - which applies to every team working on it.
- **Attribution with a verifiable axiom.** Integrated gradients, chosen because its completeness property is checkable on every call - and it is checked, in the library, in the shipped payload's tests, and again in the browser.
- **The instrument caught an error in the team's own published result.** See Section 26.

The 30-second answer to "What is your project?"

"Most attack-prediction systems are classifiers - they score the traffic they saw. We built a world model instead: it learns how network state transitions, so it can roll forward and answer a question a classifier cannot even represent - what happens if we isolate this host right now. We tested it properly, including on thirteen real botnet captures, and our forecasting claim lost to a gradient-boosted tree. We publish that. What survives is the counterfactual, and an evaluation harness rigorous enough that it caught a mistake in our own published numbers."

Before quoting any figure from this document

Five contradictions were found between the repository's documents, its results files and its shipped slides. They are catalogued with rulings in Section 26.7. One of them affects a headline number currently on the presentation.

2. Problem Statement

2.1 What the client asked for

Problem statement SIH26153 from NTRO is unusually specific - it is a research brief rather than a product request. The repository records the PS as asking for a system in which, rather than classifying traffic, "a world model learns the transition dynamics $P(S_{t+1} | S_t)$... This enables forward simulation: roll out K steps ahead and identify whether the current trajectory converges to an infiltration state, before the attacker completes the kill chain." It further asks for network state as feature vectors or graphs, sequence/GNN/latent-state models, mapping to MITRE ATT&CK stages, and explainability.

2.2 The distinguishing question

The repository frames the competitive situation in one sentence: most submissions to this problem statement will be an LSTM that emits a benign-or-malicious label and is described as a world model. The question that separates them is **"what does your model predict about state, as opposed to labels?"** Everything in the architecture exists to answer that.

2.3 Classifier versus world model

Property	Sequence classifier	World model
Learns	$P(y x_{1:t})$	$P(s_{t+1} s_t, a)$
Output	A label for now	A distribution over futures
Multi-step rollout	Not possible	Native
Counterfactual	Not possible	Native
Answers 'how long do I have?'	No	Yes
Answers 'what if I isolate this host?'	No	Yes

Table 1. The structural difference. The last two rows are the entire pitch, and they are properties a competing team cannot add late.

2.4 Why existing approaches are insufficient

Approach	What it does	What it cannot do
SIEM and UEBA (Splunk, Exabeam class)	Scores observed behaviour against a learned baseline	No forward simulation. It reports what happened; it does not forecast, and it cannot represent an intervention.
LSTM and transformer classifiers	Scores the trajectory that was actually observed	Cannot condition on an action that was never taken. Conditioned on an action it only ever saw correlated with outcomes, it learns the correlation, not the intervention.

Approach	What it does	What it cannot do
Attack graphs (BloodHound class)	Static reachability over known topology	No learned dynamics, no timing, no notion of how likely a path is to be walked, and no effect of intervening.
KALACHAKRA	Action-conditioned latent rollout, K steps ahead	Answers: what happens if we isolate this host now? <i>Its measured forecasting performance, however, does not beat the baselines - see Section 26.</i>

Table 2. Why no existing class of tool answers the counterfactual question.

2.5 Real-world consequences and the specific gap

Without a forward model, an analyst facing a suspicious host has only precedent and intuition to decide whether to isolate it - an action with real operational cost. The gap is the absence of a model that can be *asked* about an action. The project closes that gap architecturally; what it does not do is demonstrate that the resulting forecasts are more accurate than a well-tuned classifier's, and it says so.

3. Motivation and Relevance

3.1 Why this project was created

It answers SIH26153. It was also chosen as a companion to EKAGRA because roughly 60-70% of the data foundation is shared - packet and flow record schema, streaming feature extractors, dataset adapters and the traffic generator - so two problem statements are addressed by one engineering core. EKAGRA answers *what is happening now, given I can only observe*; KALACHAKRA answers *where is this going, and what if I act*.

3.2 Why the problem is relevant now

- **Volume.** 29.44 lakh incidents handled by CERT-In in 2025, up 85% since 2023. Triage capacity, not detection capacity, is the constraint.
- **Research direction.** Latent predictive (JEPA-style) world models are an active line in representation learning; applying them to network state is a legitimate research question rather than a repackaging.
- **Decision support.** The operational question in a SOC is increasingly about response actions with real cost, not about alert volume.

3.3 Potential real-world applications

- Decision support at the point where an analyst must choose whether to isolate a host.
- A simulator for training and for evaluating response policies - the repository explicitly positions the model as the simulator and a reinforcement-learning defender policy as future work.
- A benchmark and evaluation harness for anyone claiming cross-network attack forecasting. [INFERENCE: the repository does not describe itself as a public benchmark, but the harness is general and this is the project's most transferable asset.]

4. Proposed Solution

4.1 Core idea

Represent network state at time t as a **host-interaction graph** G_t . Encode it to a latent vector with a graph neural network. Learn a transition model that maps (latent, action) to the next latent. Roll that transition forward K steps with no new observations, feeding predictions back in. Score each predicted latent against MITRE ATT&CK kill-chain

stages, and read the probability of reaching LATERAL_MOVEMENT or beyond as the alerting signal.

4.2 Why a graph rather than a flat vector

The phenomena being forecast are **relational**. Lateral movement is an edge appearing between two internal hosts that have never communicated before. A flat feature vector destroys exactly that structure.

4.3 Why predict in latent space rather than reconstruct traffic

Packet-level detail is mostly noise for forecasting, and reconstruction spends model capacity on it. The objective is JEPA-style: predict a learned representation, with a stop-gradient on the target. The known failure mode of this family is latent collapse, which is why a variance-covariance regularisation term (VICReg) is included and the collapse diagnostic is reported.

4.4 Conceptual workflow

1. Windowed traffic is turned into a host-interaction graph per window.
2. A GNN encoder maps each graph to a 64-dimensional latent state.
3. A GRU carries context forward across observed windows.
4. From a chosen decision window, the transition model rolls K steps ahead, consuming an action at each step and feeding its own predictions back in.
5. Each predicted latent passes through the ATT&CK stage head.
6. Compromise risk is aggregated across the rollout as one minus the product of per-step survival probabilities.
7. Running the rollout twice - once with action 'none', once with 'isolate' - gives the counterfactual difference.

5. System Architecture

Integrated-gradients attribution runs over this whole path; the completeness residual is verified on every call

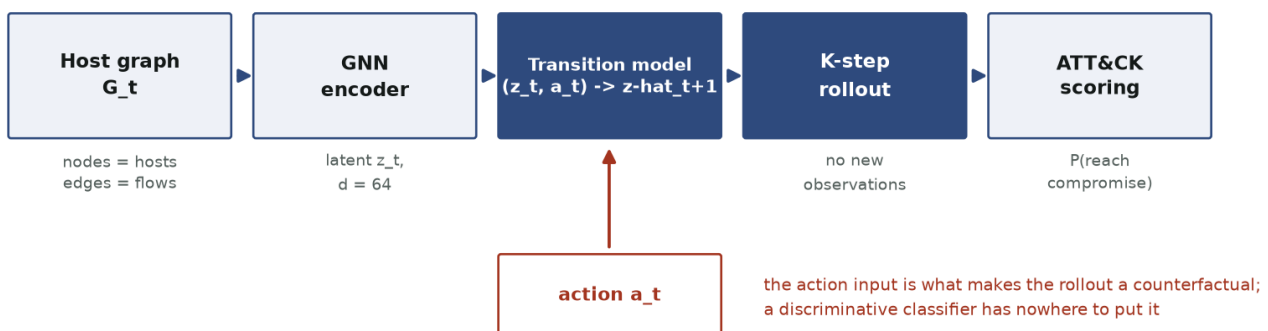


Figure 1. KALACHAKRA model architecture. The action input into the transition model is the structural feature that makes a counterfactual possible.

5.1 State representation

Element	Contents
Nodes	Hosts, internal and external. Features: packet and byte rates in and out, distinct peers, distinct ports touched, flag distribution, protocol mix, novelty score.
Edges	Communication observed in window t . Features: volume, packet count, duration, flag bitmask summary, port class.
Global	Window-level aggregates: total flows, source-IP entropy, scan-ness, new-edge fraction.

Table 3. The host-interaction graph G_t .

5.2 Model components

Component	What it is	Why it exists	Input and output
GNN encoder	Normalised-adjacency message passing	Turns a relational graph into a fixed-size latent without destroying structure	G_t in; latent z_t of dimension 64 out
GRU context	Recurrent state over observed windows	Summarises history up to t so the rollout starts from an informed state	Sequence of z in; hidden state h_t out
Transition model	Learned map (z , action) to predicted next z	The component that makes this a world model. The action input is what a classifier has nowhere to put	(z_t, a_t, h_t) in; predicted z_{t+1} out
K-step rollout	Iterated application of the transition model	Forward simulation without new observations; compounding error is something the model is optimised against rather than surprised by	Start latent and an action schedule in; K predicted latents out
ATT&CK stage head	Classifier over kill-chain stages	Grounds the latent in semantics an analyst already uses; makes the latent interpretable rather than decorative	Latent in; distribution over 7 stages out
Attribution	Integrated gradients with adaptive step refinement	Explainability with a checkable axiom	A case and a reference in; per-feature attributions plus a verified completeness residual out

Table 4. Model components as an input-processing-output chain.

5.3 The ATT&CK stage ladder

A deliberate subset, ordered as a kill chain: BENIGN, RECON, INITIAL_ACCESS, EXECUTION, LATERAL_MOVEMENT, COLLECTION, EXFILTRATION. The alerting signal is the rollout probability of reaching LATERAL_MOVEMENT or beyond, aggregated as 1 minus the product of $(1 - p_k)$ across the rollout - the attacker only has to get there once.

5.4 Training objective

Three terms. A predictive loss in latent space with a stop-gradient on the target; a cross-entropy on the ATT&CK stage, applied to both encoded and *predicted* latents; and a VICReg variance-covariance term to prevent latent collapse.

A train/use mismatch that was found and fixed

The first version trained one step ahead and supervised the ATT&CK head only on *encoded* latents. At inference the head was applied to *predicted* latents, which it had never seen. That mismatch left rollout risk near chance - AUC 0.61 against an LSTM's 0.97. Both halves are now fixed: predicted latents go through the same head during training, and the rollout is multi-step so compounding error is part of the objective.

5.5 What is deliberately not built

Not built	Reason recorded in the repository
A reinforcement-learning defender policy	The obvious next step and a research programme in itself. Cannot be evaluated honestly in the available time. The project provides the simulator; the policy is future work.
Raw-traffic generative reconstruction	Wrong objective, enormous cost.
A full SOC or SIEM	Out of scope.
Real-time inference at line rate	KALACHAKRA operates on windowed graph snapshots measured in seconds, not packets. A different latency regime from EKAGRA.

Table 5. Deliberate scope exclusions.

6. End-to-End Workflow

1. **Traffic is windowed.** Flows are bucketed into fixed windows (60 seconds for the CTU-13 adapter).
2. **A graph is built per window.** GraphBuilder produces node features, an adjacency matrix, a novelty matrix and global features for a fixed host set.
3. **Episodes are stacked.** A sequence of window graphs plus stage labels becomes an episode tensor of shape (time, nodes, features).
4. **The encoder runs over every window.** Each graph becomes a latent z_t ; a GRU carries context forward.
5. **Training rolls forward from every valid start position.** From each start, the transition model predicts R steps, feeding predictions back in; predicted latents are supervised against observed latents and through the stage head.
6. **At inference, a decision window is chosen.** The rollout runs K steps with a chosen action schedule.
7. **Compromise risk is aggregated.** 1 minus the product of per-step survival probabilities over the horizon.
8. **For a counterfactual, the rollout runs twice** - action 'none' and action 'isolate' applied at a chosen step - and the difference is the predicted effect.
9. **Attribution runs over the same path**, with the completeness residual checked and the reference state recorded.
10. **The console replays it.** Trajectories are computed once and written to a static file the browser loads directly.

K-step rollout under two actions

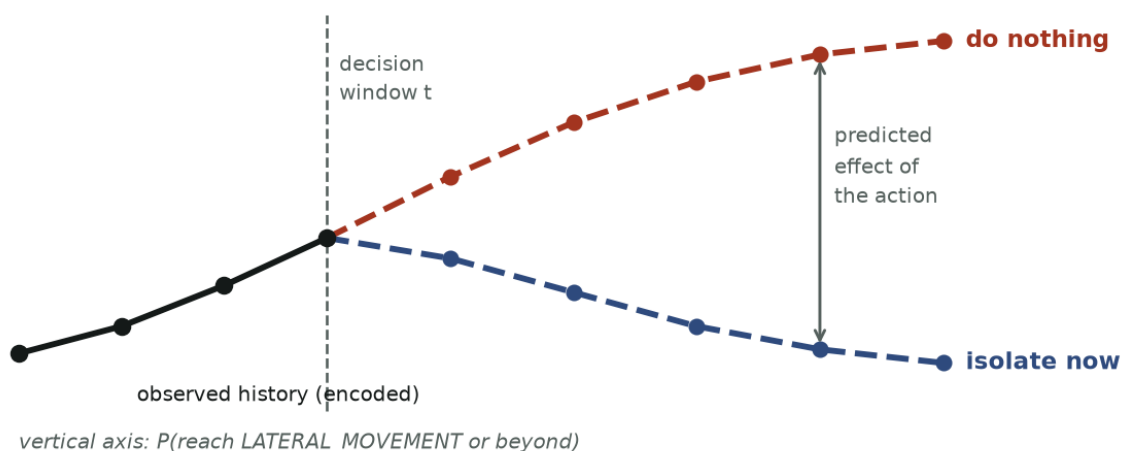


Figure 2. The counterfactual mechanism: one observed history, two rollouts under different actions, and the gap between them as the model's belief about the intervention.

7. Features

7.1 Core features

Feature	What it does / why	How it works	Output and dependencies
Graph state representation	Preserves the relational structure that lateral movement consists of	Per-window host-interaction graph with node, edge and global features	Stacked episode tensors. Depends on <code>state/graph.py</code>
Latent transition model with action input	The differentiator. Lets the model be asked about an action	(z, a, h) mapped to predicted next z; action embedded and consumed at each rollout step	Predicted latents. Depends on <code>dynamics/world_model.py</code>
K-step rollout	Forward simulation with no new observations	Iterated transition with predictions fed back; the depth used at training must match the horizon used at inference	K predicted latents and stage distributions
ATT&CK stage scoring	Makes the latent interpretable in vocabulary a SOC already uses	Stage head over 7 ordered kill-chain stages	P(stage) per predicted step; compromise risk aggregated over the horizon
Counterfactual rollout	Answers 'what if we isolate this host now?'	Two rollouts from an identical prefix differing only in the action schedule; <code>action_at</code> sets the step at which the action begins	A predicted effect size per case
Integrated-gradients attribution	Explainability the PS asks for, with an axiom that can be verified	Path integral from a reference state, with step count refined per case until the completeness residual fits the gap	Per-feature attributions plus a residual. Depends on <code>explain/attribution.py</code>
Lead-time evaluation harness	Measures what a defender actually buys: warning time at a matched false-alarm rate	Threshold sweep producing lead time, detection rate and FPR; non-compromised episodes - including contained attacks - are the negatives	Operating points. Depends on <code>metrics/leadtime.py</code>

Table 6. Core features and dependencies.

7.2 Data and evaluation features

- **Multi-stage episode generator** with ATT&CK stage labels, branch points, and a third population that matters: *contained* episodes with real attack activity that never reaches compromise.
- **CTU-13 adapter** that turns real botnet captures into the same Episode objects the generator produces, so every model runs unchanged - if the adapter built its own features, a synthetic-versus-real difference would be unattributable.
- **Matched counterfactual pairs** with bit-identical prefixes before the fork, so any divergence is caused by the action rather than by noise.
- **Collapse diagnostic** reporting latent standard deviation and effective rank, because a judge will ask whether the latent collapsed.

8. Technical Implementation

Technology	Role	Why chosen	How used
Python 3.11+	Implementation language	Ecosystem fit; declared in <code>pyproject.toml</code>	All backend code

Technology	Role	Why chosen	How used
PyTorch	Neural network framework	The model needs custom message passing, a custom rollout loop and gradient access for integrated gradients - all of which need a define-by-run framework	GNN encoder, GRU, transition model, stage head, attribution
NumPy	Numerical processing	Standard	Tensor preparation, metrics
XGBoost	Gradient-boosted baseline	The strongest tabular baseline, and the one that beats the world model on real data - included deliberately	GBDT baseline in every comparison
scikit-learn	Baselines and metrics	Logistic regression baseline; AUC, F1	Baselines and evaluation
matplotlib	Figures	Reproducible result figures	Experiment figure generation
pytest	Testing	Standard	107 tests
Plain HTML, CSS and JavaScript	Trajectory console	No build step and no CDN, matching EKAGRA's air-gapped constraint	Single <code>index.html</code> plus a <code>data.js</code> replay payload

Table 7. Technology choices with rationale.

8.1 Notable implementation details

- **The horizon is now an argument, not a module constant.** `build_dataset(..., horizon=)`, `train_world_model(..., rollout_depth=)` and `wm_scores(..., horizon=)` are passed together, so training and inference cannot silently disagree. A dedicated test module pins the defaults so no previously published number moves.
- **The world-model trainer is shared, not reimplemented.** Experiments 14 through 18 import the trainer extracted from experiment 01, so a synthetic-versus-real difference cannot be an artefact of two implementations drifting apart.
- **The action is applied per rollout step, not once.** An earlier version used a single action for the whole rollout, which made an intervention part-way through invisible - exactly the case the counterfactual asks about.

9. AI, ML and Data Science Components

9.1 Problem formulation

Two distinct tasks. **Forecasting:** a per-window binary label answering "will compromise occur within the next H windows?", written $y[\max(0, c - H) : c] = 1$ where c is the compromise window. **Counterfactual estimation:** given a matched pair of episodes sharing an identical prefix and differing only in an action, predict the effect of the action.

A subtlety that invalidates naive comparisons

The forecasting label depends on the horizon H . At $H = 2$ the model is asked a rare, hard question; at $H = 18$ a common, easy one. **AUC is therefore not comparable across horizons** - it is measured against a different target in every arm. Lead time and detection count are comparable, because they read only the compromise window and the score series. Experiment 16 reports those and deliberately publishes no AUC.

9.2 Datasets

Dataset	Role	Scale and provenance
CTU-13	Real-data validation - the only real-attack corpus here	Thirteen real botnet captures, Stratosphere Lab, CTU Prague (2011); 19.9 million flows. Obtained via a Kaggle mirror; version 1 must be used because later versions drop <code>StartTime</code> , <code>SrcAddr</code> and <code>DstAddr</code> .

Dataset	Role	Scale and provenance
In-repo episode generator	Controlled instrument	data/episodes.py, 396 lines. Produces attack, benign and <i>contained</i> episodes with ATT&CK stage labels and matched counterfactual pairs.
Matched counterfactual pairs	The only way to validate a counterfactual honestly	Prefixes are bit-identical before the fork because the RNG is re-seeded identically for both branches; 200 held-out test pairs.

Table 8. Datasets and roles.

9.3 The CTU-13 adapter and the confounds it had to remove

Turning real captures into comparable episodes required three corrections, each of which shipped as a bug first and is now pinned by a test:

Problem	Why it invalidated results	Fix
Pseudo-replication	A scenario's infected hosts wake within minutes of each other, so one episode per infected host shares almost all pre-compromise context. That multiplies n without adding evidence.	One attack episode per scenario. CTU-13 is thirteen independent infection events, not more.
A 500-times activity confound	The monitored set is quiet infected hosts plus the busiest machines on the network. Matching the clean set to the infected median produced attack episodes of 167,000 flows against benign episodes of 300 - any model could score by learning 'busy means attack'.	Both host sets are interleaved down the activity ranking, giving a 1.05x activity ratio. Two earlier fixes failed first.
Only four benign episodes	A false-alarm rate over four episodes quantises to $\{0, .25, .5, .75, 1\}$, and lead time is measured at a <i>matched false-alarm rate</i> . Four is unusable, not merely small.	Benign episodes are drawn from never-infected hosts as well as from the pre-infection period.

Table 9. Confounds removed from the CTU-13 adapter. Each is pinned by a named test in tests/test_ctu13.py.

9.4 Model architecture in detail

- **Encoder:** normalised-adjacency message passing over the host graph. **There is no attention mechanism anywhere in the model** - this matters because the PS names attention as an explainability option and claiming it would be false.
- **Latent dimension:** 64.
- **Context:** a GRU over encoded latents.
- **Transition:** a learned step function consuming latent, hidden state and an embedded action.
- **Heads:** an ATT&CK stage classifier over 7 ordered stages, applied to both encoded and predicted latents.
- **Rollout:** from every valid start position during training; R start positions are T minus R , so a deeper rollout leaves fewer starts.

9.5 Training methodology

Adam optimiser, learning rate $2e-3$. The composite loss combines the latent predictive term (MSE against a stop-gradient target), cross-entropy on rolled-out stage predictions, cross-entropy on stages from encoded latents, and a VICReg term. Training configuration is carried in a TrainConfig dataclass covering epochs, learning rate, stage and VICReg loss weights, and batch size.

9.6 Evaluation protocol

- **Temporal splits only** on generated data; **leave-one-scenario-out** on CTU-13, so every test episode comes from a network and a malware family the model has never seen. That is the strongest protocol the data supports and also

the hardest - and it is what a deployed sensor faces.

- **Baselines run on identical features and splits:** logistic regression, gradient boosting, and a tuned LSTM.
- **Lead time at a matched false-alarm rate is the headline metric.** Comparing raw lead times at different alarm rates is how a bad detector is made to look good.
- **Negatives include contained attacks.** If negatives were only attack-free episodes, a model firing on the first port scan would post a perfect false-alarm rate and huge lead time while having forecast nothing. Counting contained attacks as negatives is what forces the task to be about progression.
- **Nested selection where anything is tuned.** Configuration is chosen on inner validation captures the test fold is not part of.

9.7 Explainability - what is shipped and what is substituted

The PS names SHAP values or attention. The project has neither and says so: the encoder is message passing with no attention to read off, and SHAP is a dependency plus thousands of forward passes per explanation. **Integrated gradients** is shipped instead, and the argument is one property rather than a preference: **completeness**, the guarantee that attributions sum to $f(\text{input}) - f(\text{baseline})$. That is the same axiom SHAP's efficiency property provides, and unlike SHAP it is cheap enough to verify on *every call* - which is done in the library, in the shipped payload's tests, and again in the browser.

Design decision	Why it is load-bearing
The reference is a quiet network, not zeros	Measured: an empty graph scores 0.768 compromise risk on the trained model, and zeros with zero adjacency score 0.980 - the model finds 'nothing is happening' alarming. Against zeros we would be explaining only the sliver above 0.77, referenced to a state the network is never in. Against the benign mean (0.027), attributions read as <i>what makes this different from a normal day</i> .
The step count adapts per case	A flat 32 steps left residuals near $1e-2$ - negligible against a gap of 0.97, but larger than the whole gap on a quiet case, where the sum came out with the wrong sign. Refining until the residual fits the gap gives 40 of 40 converged at 52 steps on average.
Every result records its reference	An attribution whose reference is hidden is not an explanation.

Table 10. Attribution design decisions.

The limit is stated wherever attributions appear: this explains **the model's function, not the network**. A learned spurious correlate is reported faithfully - which is true of SHAP too.

10. Data Stores and Data Flow

There is no database

KALACHAKRA uses no database, no ORM and no storage service. Inputs are files; intermediate state is in memory or a pickle cache; output is a static JavaScript payload. The console is a replay of a finished run, not a live system - and the repository is explicit that a server would imply otherwise.

Store	Form	Contents and lifecycle
CTU-13 source data	CSV (Argus binetflow exports)	Thirteen scenario files. Read once per cache build.
Episode cache	Pickle, results/ctu13_episode_full.pkl	Parsed episodes grouped by scenario. Cache key includes quick mode, because silently reusing a capped cache for a full run would report a real-looking result from 400k rows per scenario instead of 19.9M.
Trained model	PyTorch state dict, results/demo_model.pt	The demo model backing the console replay.
Verdict files	JSON, results/exp*_verdicts.json	Pre-registered predictions and their measured verdicts. Written by the run itself , which is what makes them authoritative over prose.

Store	Form	Contents and lifecycle
Console payload	frontend/data.js	One global, <code>window.KALACHAKRA_DATA</code> , containing generated timestamp, horizon, the stage list, the compromise stage and the trajectories.

Table 11. Data stores and lifecycles.

10.1 Data validation

Validation is structural and refuses rather than guesses: unparseable timestamps raise a `ValueError` rather than defaulting; Argus state flags are read from the source half only, because on a one-way tap the destination half is not observable and reading it would credit the model with evidence the sensor cannot have; episodes are required to be of uniform length, since a ragged episode would otherwise fail far away with a shape error rather than a diagnosis.

11. APIs and Integrations

There is no runtime API and there are no external integrations

KALACHAKRA ships no HTTP API. The console loads a static `data.js` with a plain script tag and works when opened directly from the filesystem. There are no third-party services, no API keys, no tokens and no credentials anywhere in the system.

11.1 The one external data dependency

CTU-13 is fetched once by `fetch_ctu13.py` from a Kaggle mirror, using the user's own Kaggle credentials file. This is a one-time data acquisition, not a runtime integration - once the cache exists, nothing reaches the network. The script pins **version 1** deliberately: later versions of that mirror drop the `StartTime`, `SrcAddr` and `DstAddr` columns, without which the adapter cannot build episodes at all.

11.2 Internal interfaces that behave like contracts

Interface	Contract
<code>window.KALACHAKRA_DATA</code>	generated, horizon, stages[], compromise_stage, trajectories - the only coupling between the model and the console
Episode objects	The CTU-13 adapter emits exactly what the generator emits, so <code>GraphBuilder</code> and every model run unchanged on real and generated data
<code>train_world_model(...)</code>	Shared by experiments 01 and 14-18, so all of them provably optimise the same objective
<code>lead_at_fpr(scores, budget)</code>	The single definition of the operating point used by every reported comparison

Table 12. Internal interfaces.

12. User Journey

1. The analyst opens the console - a single HTML file, no server required.
2. They select a case: a held-out episode at its decision window.
3. The console shows the observed history up to that window.
4. Two forward trajectories are drawn from the same prefix: 'do nothing' and 'isolate now'.
5. The gap between the curves is the model's predicted effect of the intervention.
6. The realised outcome of both branches is shown underneath, because these are matched pairs where the ground truth of both futures is known.

7. The analyst opens the attribution panel to see which host attributes drove the forecast.
8. The console verifies the completeness residual in the browser before it will call a ranking trustworthy.

13. User Interface and Experience

A single `index.html` of roughly 21 KB with a `data.js` payload of roughly 61 KB. No framework, no build step, no CDN, no server.

Component	Purpose	What the user sees
Case selector	Choose which held-out episode to inspect	A list of cases; selecting one loads its trajectories
Trajectory view	The demonstration moment. Show the two futures separating	Observed history, then two forward curves - 'do nothing' and 'isolate now' - diverging from a shared prefix
Realised outcome	Ground truth for both branches	What actually happened in each branch of the matched pair
Attribution panel	Explain which attributes drove the forecast	Ranked per-feature attributions with the reference state named and the completeness residual shown

Table 13. Console components.

An honesty decision embedded in the UI

The console is explicitly a *replay* of a finished model run, not a live system. The repository states the reason plainly: nothing here is live, and running a server would imply it was. If the first run has no `data.js`, `demo.py` computes it - about seven minutes of training.

14. Security and Privacy

Stated plainly: there is no authentication and no authorisation

There is no login, no accounts, no session management and no access control. The attack surface is correspondingly small - the console is a static file with no server and no network calls - but nothing restricts who may open it if they have the file.

Property	Status
Network exposure at runtime	None. The console makes no network requests; it reads a local script file.
Credentials in the system	None, except the user's own Kaggle credentials file used once for data acquisition. That file is the user's and is not part of the repository.
Input validation	Structural and refusing: unparseable timestamps raise; only the source half of Argus state flags is read; episode length uniformity is enforced.
Data sensitivity	CTU-13 is a public research corpus. No private or personal data is processed.
Model security	Not addressed. There is no defence against adversarial examples, model extraction or data poisoning, and none is claimed. [INFERENCE: no such mechanism appears in the code or documentation.]
Supply chain	Five direct dependencies: numpy, torch, scikit-learn, xgboost, matplotlib. No pinned lockfile is present. [INFERENCE]

Table 14. Security posture, stated without inflation.

15. Performance and Scalability

KALACHAKRA operates on **windowed graph snapshots measured in seconds, not on packets**. It is a different latency regime from EKAGRA and the repository says so explicitly rather than implying line-rate operation.

Measurement	Value	Source
World-model training, 107 episodes, 12 epochs	approximately 15 s	Measured during the horizon-sweep work
Deeper rollouts are cheaper, not costlier	Rollout depth 18 trained faster than depth 6, because start positions number T minus R	Measured during the same work
Per-window scoring	under 1 s for a batch of episodes	Measured
Console demo generation	approximately 7 minutes if data.js is absent	frontend/README.md
Full horizon sweep (exp 16)	1,419 s	exp16 run log
Seed-variance study (exp 17)	624 s	exp17 run log
Training-length sweep (exp 18)	1,775 s	exp18 run log
Test suite	107 tests in approximately 15 s	Measured

Table 15. Measured performance figures.

15.1 Scalability considerations

- **Graph size is fixed per configuration** - the CTU-13 adapter uses 24 hosts. Scaling to thousands of hosts is not demonstrated and the message passing is dense. [INFERENCE: dense adjacency implies $O(N^2)$ per layer; no sparse implementation is present.]
- **Training is CPU-bound in practice** at the sizes used; no GPU requirement is documented.
- **The real constraint is not compute, it is data.** With twelve scored attack episodes, the variance between captures dwarfs the difference between model configurations - which is the finding, not a complaint.

16. Deployment

16.1 Environment

Python 3.11 or later with numpy, torch, scikit-learn, xgboost and matplotlib, as declared in `kalachakra/backend/pyproject.toml`. Package layout is `src/` with pytest configured for `pythonpath = ["src"]`.

16.2 Running it

```
cd kalachakra/backend
python demo.py # builds data.js if absent (~7 min), then
# the console opens from the filesystem

python fetch_ctu13.py --version 1 # one-time CTU-13 acquisition
# version 1 is required

python -m pytest tests/ -q # 107 tests
python experiments/exp14_ctu13_leadtime.py # real-data evaluation
python experiments/exp16_ctu13_horizon.py # horizon sweep (~24 min)
python experiments/exp18_ctu13_epochs.py # training-length sweep
```

16.3 Production posture

- **There is no production deployment path.** No container, no service manifest, no server, no CI/CD. The console is a static file.

- **No environment variables and no secrets**, other than the user's own Kaggle credentials for the one-time data download.
- [INFERENCE] Deploying this as a live decision-support tool would require building an inference service, a streaming graph builder and an authenticated interface - none of which exist. What exists is the model, the evaluation and a replay console.

17. Project and Codebase Structure

```

kalachakra/
ARCHITECTURE.md design rationale and the research argument
RESULTS.md full experiment log (807+ lines)
backend/
pyproject.toml deps, pytest config
demo.py builds the console payload
fetch_ctu13.py one-time CTU-13 acquisition (--version 1)
src/kalachakra/
data/ episodes.py (generator), ctu13.py (adapter)
state/ graph.py (GraphBuilder, stacking)
dynamics/ world_model.py (encoder, transition, rollout)
metrics/ leadtime.py (the headline metric)
explain/ attribution.py (integrated gradients)
experiments/ exp01, exp02, exp10-exp18
tests/ 8 modules, 107 tests
results/ verdicts JSON, caches, demo payload
frontend/
index.html the console (~21 KB)
data.js replay payload (~61 KB)
README.md the data contract

```

File	Lines	Purpose
dynamics/world_model.py	428	The model: GNN encoder, GRU context, action-conditioned transition, K-step rollout, ATT&CK head, compromise-risk aggregation, VICReg and the collapse diagnostic.
data/episodes.py	396	Episode generator: attack, benign and contained populations, stage labels, matched counterfactual pairs with bit-identical prefixes.
data/ctu13.py	387	Real-capture adapter. Contains the activity-interleaving host selection that removed the 500x confound, and the one-episode-per-scenario rule.
explain/attribution.py	228	Integrated gradients with adaptive step refinement and completeness verification.
state/graph.py	195	GraphBuilder: window graphs, node and global features, episode stacking.
metrics/leadtime.py	136	Lead time, false-alarm rate, detection rate and the operating-point selection every comparison uses.

Table 16. Principal source files.

17.1 Experiment inventory

Experiment	Question it asks	Predictions	Refuted
exp01	Lead time against LSTM, GBDT and logistic regression (generated data)	4	2
exp02	Counterfactual validation on matched pairs	4	2
exp10	Does FiLM conditioning recover a per-host effect?	4	3
exp11	Does a paired-difference loss recover it?	4	1
exp12	Oracle diagnostic - trained on the target itself	diagnostic, excluded from the count	-
exp13	Demo trajectory payload	none	-

Experiment	Question it asks	Predictions	Refuted
exp14	Lead time on CTU-13 real captures	6	1
exp15	Does honest nested tuning help?	4	1
exp16	Does the rollout horizon explain the result?	5	4
exp17	Is the published number a seed artefact?	4	2
exp18	Is it a training-length artefact?	5	0
Total		40	16

Table 17. Pre-registered predictions per experiment, computed from the verdict files during authoring.

18. Algorithms and Core Logic

18.1 The JEPA-style latent predictive objective

Problem. Learn dynamics without spending capacity on reconstructing packet detail. **Approach.** Predict the *latent* of the next window rather than its raw features, with a stop-gradient on the target so the encoder cannot cheat by collapsing. **Failure mode.** Latent collapse - the encoder maps everything to a constant, making the prediction trivially perfect. **Guard.** A VICReg variance-covariance term, plus a reported diagnostic: measured latent standard deviation 0.975 with effective rank 24.65 of 64 dimensions in experiment 01, and 1.078 with effective rank 41.11 in experiment 02. The latent did not collapse.

18.2 Multi-step rollout with prediction feedback

Problem. A one-step-trained model applied K steps ahead accumulates error it was never optimised against. **Approach.** From every valid start position, roll R steps forward, feeding predicted latents back in as inputs, and supervise the predicted latents through the same ATT&CK head that inference uses. **Constraint.** The number of start positions is T minus R, so a deeper rollout gives fewer training positions - which is why deeper rollouts trained faster. **Why it matters.** Not doing this was the bug that left rollout risk near chance at AUC 0.61.

18.3 Compromise-risk aggregation

Problem. Convert K per-step stage distributions into one alerting number. **Approach.** Sum the probability mass at or beyond LATERAL_MOVEMENT for each step to get p_k , then aggregate as $1 - \text{product of } (1 - p_k)$. **Rationale.** The attacker only has to get there once, so the correct aggregation is the complement of surviving every step. **Consequence:** risk is monotonically non-decreasing in the horizon, which is why comparing raw scores across horizons is meaningless and only the operating point is comparable.

18.4 Lead time at a matched false-alarm rate

Problem. Measure what a defender actually buys. **Definitions.** For an attack episode with compromise window c and per-window risk s_t : $\text{alarm}(\tau)$ is the first t where s_t is at or above τ ; $\text{lead}(\tau)$ is c minus $\text{alarm}(\tau)$ if the alarm precedes compromise, and zero otherwise. Lead is clamped at zero because firing after the attacker has moved laterally is worth nothing, and allowing it negative would let late alarms subsidise the mean. **False alarms** are counted per episode, not per window - an analyst who gets one spurious page has been interrupted once. **The critical definition:** a negative is any episode that never reaches compromise, *including contained attacks*. If negatives were only attack-free episodes, a model that fires on the first port scan would post a perfect false-alarm rate and huge lead time while having forecast nothing.

18.5 Integrated gradients with adaptive refinement

Problem. Attribute a forecast to input features with a guarantee. **Approach.** Integrate the gradient along a straight path from a reference state to the input; by construction the attributions sum to $f(\text{input}) - f(\text{reference})$. **Verification.**

That sum is compared to the actual gap on every call; the step count is refined - up to 2,048 steps - until the residual fits the gap. **Defect this fixed:** at a flat 32 steps, a case with a gap of +0.0077 produced an attribution sum of -0.1667 - the wrong sign entirely. Adaptive refinement gives 40 of 40 cases converged at an average of 52 steps.

18.6 Nested selection - the protocol that prevents self-deception

Problem. Tuning under leave-one-scenario-out and keeping the best configuration *is tuning on the test set*. With twelve attack episodes and four configurations, keeping the luckiest would very plausibly turn 1 of 12 into 3 of 12 out of pure search. **Approach.** For each outer fold, the test scenario is held out entirely; a configuration is chosen on a separate inner validation capture, rotated so it is never the same one; the model is then retrained on all non-test scenarios with that choice and scored once. **Guarantee:** no reported number has seen its own test fold. **Cost:** the winning configuration differs between folds, so there is no single tuned model to ship - which is itself reported.

19. Innovation and Differentiation

- **A capability that is structurally unavailable to the competition.** The transition model takes an action. A discriminative model has nowhere to put one, and there is no bolt-on that adds a counterfactual to a classifier. A competing team cannot add this late.
- **Latent-space prediction rather than traffic reconstruction**, with the collapse diagnostic reported because it is the known failure mode of that family.
- **An evaluation instrument that is arguably the stronger deliverable.** Lead time at a matched false-alarm rate, leave-one-scenario-out over thirteen real captures, contained attacks counted as negatives, and pre-registered predictions with verdicts printed by the runs.
- **Attribution chosen for a verifiable axiom rather than for popularity** - and the honesty of naming that it is *not* the SHAP or attention the PS asks for, with the reason.
- **The instrument caught an error in the project's own published result.** That is the strongest test an evaluation harness can pass, and it is documented in Section 26.

Claims deliberately not made

No claim is made that the world model forecasts better than a classifier. On this project's own evidence it does not. Anyone presenting this work should lead with the mechanism and the evaluation, and volunteer the refutation.

20. Existing Solutions Compared With KALACHAKRA

Verified facts are marked [V]; qualitative analysis is marked [A].

Dimension	SIEM / UEBA	LSTM / transformer classifier	Attack graphs	KALACHAKRA
Forecasts future state	[A] No	[A] Partially - scores the observed trajectory	[A] Static reachability only	[V] Yes - K-step rollout
Answers a counterfactual	[A] No	[A] No - structurally	[A] No	[V] Yes
Detection quality on real botnet data	[A] Not measured here	[V] LSTM: 4 of 12 caught	[A] Not applicable	[V] 5 of 12 - loses to GBDT's 7 to 9
Explainability	[A] Rule provenance	[A] Attention if present	[A] Path enumeration	[V] Integrated gradients with verified completeness
Maturity	[A] Production, years	[A] Well understood	[A] Production tooling	[A] Research prototype. No deployment path
Cost	[A] Commercial licence	[A] Free to build	[A] Mixed	[V] Zero

Table 18. Comparison. KALACHAKRA wins on capability class and loses on measured forecasting performance - both are stated.

21. Use Cases

21.1 Primary - intervention decision support

Element	Detail
User	SOC analyst at a CERT-In empanelled or NCIIIP-designated centre
Situation	A host is behaving suspiciously; isolating it has real operational cost
Problem	Detection scores say something is happening; they cannot say whether acting now changes the outcome
How it is used	Select the host at its decision window; read the two forward trajectories
Expected result	A predicted effect of isolation, with the attributions that drove the forecast
Benefit	The decision is informed by a model of dynamics rather than by precedent alone
Honest caveat	The model predicts an average effect, not a per-host one. It can indicate that isolating helps in general; it cannot yet rank which of two hosts to isolate first. Four experiments established this.

Table 19. Primary use case, including the limit that constrains it.

21.2 Secondary use cases

- **Evaluation harness for attack forecasting.** Any team claiming cross-network forecasting can be asked for lead time at a matched false-alarm rate under leave-one-scenario-out with contained attacks as negatives. [INFERENCE: the repository does not market this, but it is general.]
- **Simulator for response-policy research.** The repository explicitly positions an RL defender policy as future work built on this simulator.
- **Attack-progression triage vocabulary.** Rollout output maps onto MITRE ATT&CK stages a SOC already uses.
- **A worked example of experimental discipline.** Pre-registration, deterministic controls, nested selection and self-correction, applied to a security ML problem.

22. Advantages

Category	Advantage
Technical	Answers a question class that classifiers cannot represent. Latent prediction avoids spending capacity on reconstruction, with collapse measured rather than assumed away (effective rank 24.65 to 41.11 of 64).
Explainability	Integrated gradients with completeness verified on every call, in the library, in the shipped payload's tests and in the browser. The reference state is recorded with every result.
Evaluative	40 pre-registered predictions with 16 refuted; verdicts written by the runs. Nested selection means no reported number has seen its own test fold. Deterministic controls make discrepancies attributable.
Operational	Output is expressed in MITRE ATT&CK stages. The console runs air-gapped with no server and no network calls.
Engineering	One core serves two problem statements - roughly 60-70% of the data foundation is shared with EKAGRA.
Scientific	Produces a finding about the <i>problem</i> : on CTU-13 all four methods span roughly 0.03 to 0.99 AUC across captures, so cross-network transfer is close to a coin flip per capture. That applies to everyone working on this corpus.

Table 20. Advantages by category.

23. Limitations and Current Gaps

This section is the most important one in the document

KALACHAKRA's headline capability claim is refuted by its own evidence. Presenting this project without leading on that is both dishonest and tactically worse - a judge who discovers it unprompted has caught the team out.

23.1 Capability limitations

- **Lead time: REFUTED, twice.** On generated data a tuned LSTM beat the world model 3.35 windows to 0.26 at a false-alarm rate at or below 10%. On CTU-13 the refutation replicated.
- **Window-level F1: loses to logistic regression** - 0.412 against 0.606 on generated data.
- **On real botnet captures it catches 5 of 12 infections; gradient boosting catches 7 to 9.**
- **No per-host conditional effect, and the limit is structural.** Four attempts - more data, FiLM conditioning, a paired-difference loss, and an oracle trained on the target itself - gave correlations with true containment probability of -0.105, -0.103, +0.037 and +0.011. Four nulls wandering across zero. The model predicts an average effect, not a per-host one.
- **The counterfactual headline is beaten by a trivial baseline.** Direction accuracy measures how often the model says isolating helps. A constant 'always isolate' predictor scores 1.000 by construction. The number is evidence about the LSTM's failure, not about ranking ability, and it is qualified everywhere it appears.
- **Cross-network transfer is near chance per capture** for every method including the baselines.
- **The rollout horizon does not help this model.** Sweeping it, lead time *falls* with horizon ($r = -0.842$) while gradient boosting's rises monotonically from 5 to 9 of 12. Widening the forecasting window is a task change that helps everyone, and it helps the world model least - the opposite of what a model premised on multi-step rollout should show.

23.2 What could not be tested

- **The counterfactual on real data.** No real capture contains the branch that did not happen. The intervention claim is validated on constructed matched pairs only, and always will be without a controlled experiment on a live network.
- **Intermediate ATT&CK stages on real data.** CTU-13 labels flows botnet, normal or background with no kill-chain ladder, so the stage head degenerates to compromised-or-not there.

23.3 Engineering gaps

- No deployment path: no inference service, no streaming graph builder, no authenticated interface. The console is a replay.
- Graph size fixed at 24 hosts for the CTU-13 configuration; scaling not demonstrated.
- No adversarial robustness work of any kind.
- No dependency lockfile. [INFERENCE]

23.4 Documentation gaps

ARCHITECTURE.md section 3 still states that "the action embedding is untrained" and that counterfactual rollout returns the same trajectory for every action. That was true when written and is now stale - experiments 02, 10 and 11 train it. See Section 26.7.

24. Future Scope

24.1 Short term

Item	Why	Feasibility
Reframe the deliverable around the evaluation instrument	The model does not beat the baselines; the harness is genuinely novel and demonstrably works - it caught an error in the project's own published result. This raises the project's standing without any new science.	High - documentation only
A pairwise-ranking counterfactual metric	Direction accuracy is beaten by a constant predictor. A paired-ranking metric, where a constant scores 0.5 by construction, is the well-posed version of the test.	High - but expect a null, since the correlation tests already failed four ways
Reconcile ARCHITECTURE.md with the code	The stale 'action embedding is untrained' paragraph directly contradicts three experiments.	High
Record every knob in every verdicts payload	exp14 did not record its epoch count and so could not regenerate its own published row. Partially fixed; should be systematic.	High

Table 21. Short-term roadmap.

24.2 Medium term

- **A different formulation for the conditional effect.** Four attempts at recovering a per-host effect failed, and the oracle diagnostic says the limit is structural rather than sample size. A counterfactual computed as the difference of two rollout risks collapses to an average regardless of what supervises it - so the fix is a different formulation, not another loss term.
- **More real corpora.** CTU-13 is from 2011. Modern botnet captures would test whether the near-chance transfer result is about the corpus or about the problem.
- **Sparse graph operations and larger host sets**, to test whether the approach scales beyond 24 nodes.
- **An inference service**, if the project is ever to be more than an evaluation and a replay.

24.3 Long term

- **A reinforcement-learning defender policy** built on the simulator - explicitly named as the intended next step, and explicitly excluded from this submission because it cannot be evaluated honestly in the time.
- **A public benchmark** for cross-network attack forecasting, built around the lead-time-at-matched-FPR protocol. On the current evidence the field needs one: every method tested spans 0.03 to 0.99 AUC across captures.
- **Controlled live-network intervention studies**, the only way to validate a counterfactual on real traffic.

25. Testing and Validation

107 tests across 8 modules, counted from the repository during authoring.

Test module	What it protects
test_ctu13.py	17 tests. Every test that names a bug pins one that actually shipped: comparable activity between host sets, one attack episode per scenario, benign episodes existing and correctly labelled, Argus flags read from the source half only, refusal of unparseable timestamps.
test_world_model.py	Encoder, transition, rollout shapes; the collapse diagnostic.

Test module	What it protects
test_horizon_param.py	7 tests pinning the horizon parameterisation: the published defaults are unchanged, a longer horizon labels strictly more windows, the horizon cannot label past the compromise window, a deeper rollout leaves fewer start positions, and scoring at a different horizon actually changes the scores.
test_leadtime.py	Lead-time definitions, clamping at zero, per-episode false-alarm counting, and the operating-point selection.
test_attribution.py	Completeness residual verification and reference handling.
test_graph.py	Graph construction and episode stacking.
test_episodes.py	Generator populations, stage monotonicity, matched-pair prefix identity.
test_demo_payload.py	The console payload contract, including in-payload completeness checks.

Table 22. Test modules.

25.1 Validation methodology

Pre-registered predictions written into each experiment file before the first run; verdicts printed by the run and persisted to JSON; predictions never edited afterwards. **40 predictions, 16 refuted.** Two further devices are used where they matter:

- **Deterministic controls.** Gradient boosting has no training randomness on this data, so it serves as a control: if it reproduces a previous run exactly, any difference must lie in the world-model path. This is what made the Section 26 correction possible.
- **Diagnostics are excluded from the prediction count.** Experiment 12 trains on the evaluation target deliberately, to distinguish 'not enough signal' from 'structurally impossible'. It is flagged in the file, excluded from the tally, and its payload carries an explicit warning that it is not a performance claim.

26. Results and Evidence

26.1 Generated-data results (experiments 01 and 02)

Metric	Logistic regression	GBDT	LSTM	World model
Window-level AUC	0.9378	0.9210	0.9678	0.7840
Window-level F1 (test)	0.6061	0.5163	0.7115	0.4118
Counterfactual direction accuracy	-	-	0.48 (action-aware)	0.82

Table 23. Generated-data comparison. Sources: exp01_leadtime.json, exp02_counterfactual.json. The world model is last or near-last on every discriminative metric.

Latent collapse diagnostic: mean standard deviation 0.975 with effective rank 24.65 of 64 in experiment 01, and 1.078 with effective rank 41.11 in experiment 02. The latent did not collapse - the poor discriminative performance is not a collapse artefact.

26.2 Real-data results on CTU-13 (experiment 14)

Method	AUC	sd	F1	Caught	Mean lead	FPR
World model	0.663	0.330	0.101	1 / 12	2.50	0.020
LSTM	0.647	0.305	0.015	4 / 12	8.67	0.059
GBDT	0.624	0.322	0.132	7 / 12	8.08	0.088
Logistic regression	0.703	0.341	0.203	1 / 12	1.58	0.078

Table 24. Experiment 14 as originally published, from exp14_verdicts.json. 13 scenarios, 118 episodes, 19.9M flows, leave-one-scenario-out. **The world-model row was later found to be irreproducible - see 26.4.**

26.3 The horizon sweep (experiment 16)

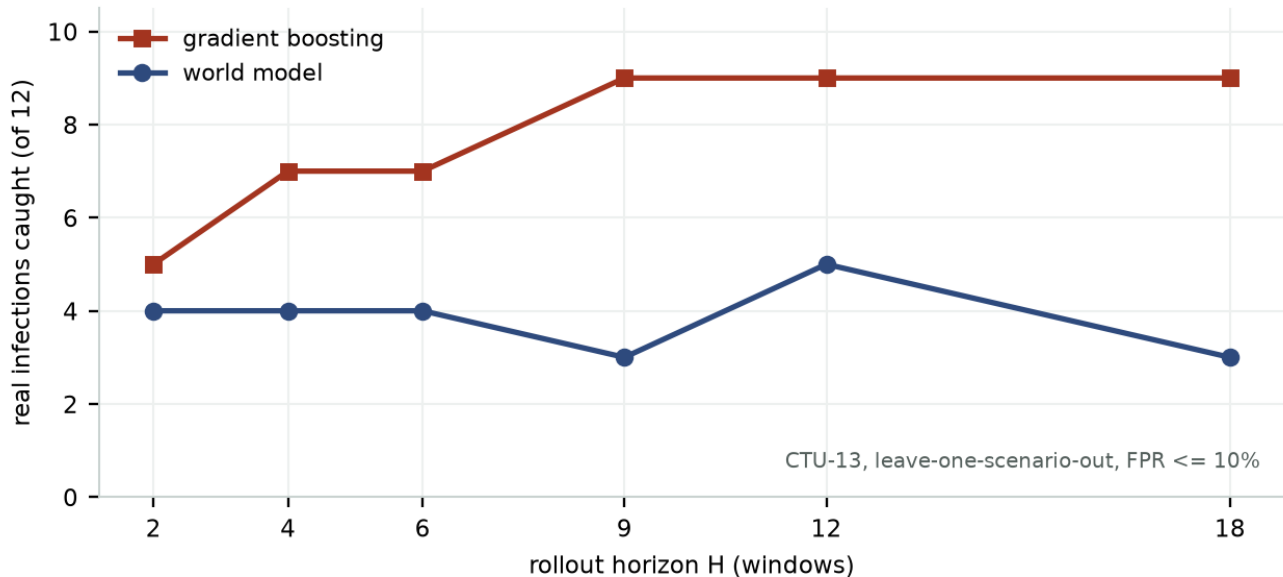


Figure 3. Infections caught against rollout horizon. Gradient boosting improves monotonically with a wider forecasting window; the world model does not.

Even allowed to pick its horizon *knowing the test results* - which nothing deployable can do - the world model reaches 5 of 12 against gradient boosting's 9. The upper bound loses, which closes the question: no honest procedure could have rescued it. The honestly-selected result, with the horizon chosen on inner validation captures only, is also 5 of 12 at mean lead 9.00 and FPR 0.069.

26.4 The correction: our own published number was wrong

Experiment 16, running experiment 14's configuration on experiment 14's folds, caught **4 of 12** rather than the published 1 of 12. The investigation that followed is the strongest evidence in this project that its evaluation works.

1. **A deterministic control isolated the fault.** In the same run, on the same folds, gradient boosting reproduced experiment 14's row to the digit - 7 of 12 caught, mean lead 8.08, AUC 0.624. So the episode construction, labels, features, fold set, pooling and operating-point rule were provably identical, and the difference lay in the world-model path alone.
2. **Seed variance was ruled out (experiment 17).** Six seeds, and the world model caught 4 of 12 on every one with zero variance. Two pre-registered predictions failed as a result, correctly.
3. **The cause was training length (experiment 18).** Experiment 14's script takes an epochs argument, defaults to 12, and never recorded which value produced the published row.

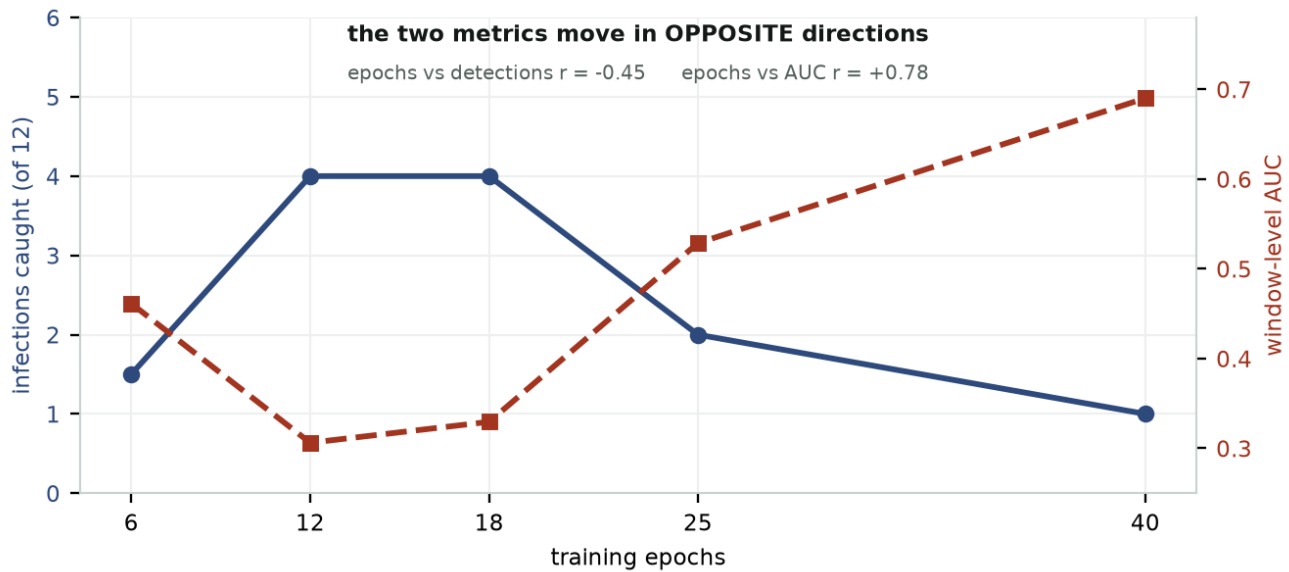


Figure 4. The finding that explains everything: training longer raises AUC and lowers the detection count. The two metrics move in opposite directions.

Epochs	Infections caught (of 12)	Mean lead	Window-level AUC
6	1.5	3.75	0.461
12	4.0	8.92	0.306
18	4.0	8.92	0.330
25	2.0	4.46	0.529
40	1.0	2.25	0.690

Table 25. Experiment 18, means over two seeds. Gradient boosting is a flat 7 of 12 in every row. epochs vs detections $r = -0.451$; epochs vs AUC $r = +0.780$.

Why this matters beyond the number

Experiment 15 - the tuning pass - selected configurations **by AUC**. AUC rewards longer training. Longer training destroys detection. So the tuning pass was steering away from the metric actually being reported, and its grid started at 25 epochs and only went up, placing the region that works (12 to 18) outside the search entirely. Experiment 15's *conclusion* was right; its *method* was pointed backwards. The corrected honest figure is **5 of 12**, and the error had made the project look **worse** than it was.

26.5 Measured versus projected

Every number in this section is measured and reproducible. Nothing in this project's results is projected or extrapolated. The only non-measured figures used in the project's materials are the market-context statistics attributed to CERT-In and to published market reports, which are cited rather than derived.

26.6 Contradictions found between sources, and the ruling on each

Results JSON files are treated as authoritative because they are written by the runs themselves; prose documents rank second and slides third.

#	Contradiction	Ruling
D1	Counterfactual direction accuracy. The slides show 94% for the world model against 48% for an action-aware LSTM. exp02_counterfactual.json records 0.82 for the world model and 0.48 for the LSTM in the same run. RESULTS.md prose says 83% in one place. The 94% figure is from exp11's effect-supervised arm.	The slide compares across experiments. 0.94 comes from exp11's paired-difference-loss variant; 0.48 comes from exp02's LSTM, which exp11 never re-ran. The same-run, like-for-like pair is 0.82 against 0.48 . The 94% is documented in README and RESULTS.md and is not fabricated, but presenting it beside a baseline from a different experiment is not like-for-like. Recommended: use 82% vs 48%, or state explicitly that 94% is a later variant. Note also that exp11's own primary prediction (AA1) failed.
D2	ARCHITECTURE.md section 3 states the action embedding is untrained and that counterfactual rollout returns an identical trajectory for every action.	Stale. Experiments 02, 10 and 11 all train it and report non-trivial direction accuracy. The document predates them and was not updated.
D3	ARCHITECTURE.md section 3 describes Claim 2 as "the surviving differentiator, untested".	Superseded. It has since been tested (exp02) and stress-tested four ways (exp10, exp11, exp12). The current status is 'holds narrowly, with a trivial baseline scoring higher on the headline metric'.
D4	RESULTS.md carries the experiment 14 world-model row (1 of 12) as a headline in one section and the corrected figure (5 of 12) in another.	Deliberate and correctly handled. The original section is annotated as superseded rather than rewritten, which preserves the record. The correct figure to quote is 5 of 12 .
D5	Prediction counts. Various documents have carried different totals over time.	Recomputed from the verdict files during authoring: 40 predictions, 16 refuted for KALACHAKRA (82 and 33 across both projects). Experiments 12 and 13 contribute no predictions.

Table 26. Contradictions and rulings.

27. Risks and Challenges

Risk	Type	Mitigation
The core capability claim is refuted	Technical / positioning	Lead with the mechanism and the evaluation instrument rather than the forecasting claim. Volunteer the refutation - it converts a vulnerability into the credibility of everything else.
A judge finds the 94% versus 82% discrepancy	Presentation	Section 26.7 documents it with a ruling. Fix the slide or state the provenance explicitly.
Only twelve scored attack episodes	Data	Acknowledged as the binding constraint. Nested selection prevents over-fitting the selection; the variance is reported rather than smoothed.
CTU-13 is from 2011	Data	Stated. Modern captures are medium-term future work.
The counterfactual can never be validated on real data	Structural	Stated up front rather than discovered. Validation is on matched pairs where both branches are known.
Cross-network transfer near chance	Technical	Reported as a finding about the problem, applying to all four methods including the baselines.
Reported numbers drifting between documents	Operational	The cause is now understood - unrecorded configuration. Verdicts payloads now record the knobs. Section 26.7 catalogues what was found.
No deployment path	Adoption	Stated plainly. The deliverable is a model, an evaluation and a replay console, not a product.

Table 27. Risks with mitigations.

28. Presentation Knowledge Base

Scoring context. SIH evaluators spend two to three minutes per submission. Weighting: innovation and uniqueness 25%, problem understanding and clarity 20%, technical feasibility 20%, impact and scalability 20%, presentation quality 15%. The official template caps the idea submission at six slides including the title and requires PDF.

The single most important presentation decision for this project

Refutations belong on the Feasibility slide, which is where the official template asks for risks and challenges - not on the idea slide, which is where innovation is scored. Nothing is hidden by this: the trivial baseline still appears on the idea slide's chart. But the highest-value slide should carry what the system uniquely does, not what it failed to beat.

28.1 Slide 1: Title

Element	Content
Objective	Identify the submission.
Points on the slide	PS ID SIH26153; PS title verbatim; NTRO; theme Blockchain and Cybersecurity; category Software; team AlgoRhythms; team ID.
Technical detail that earns credit	Nothing technical.
Suggested visual	Typographic hierarchy only.
What the presenter says	"Problem statement SIH26153 - network attack forecasting for NTRO."
Keep off this slide	No abstract.

28.2 Slide 2: Problem

Element	Content
Objective	Establish that forecasting is a different task from detection.
Points on the slide	A SOC does not need to know that something is happening - it needs to know how long is left and whether acting helps. Detection scores answer neither.
Technical detail that earns credit	$P(S_{t+1} S_t, \text{action})$ versus $P(y x)$. Write it out.
Suggested visual	A timeline with a decision point, not a network diagram.
What the presenter says	"By the time a classifier fires, the analyst usually already knows."
Keep off this slide	Do not describe the architecture here.

28.3 Slide 3: Motivation

Element	Content
Objective	Show relevance and scale.
Points on the slide	29.44 lakh CERT-In incidents in 2025, up 85% since 2023. At that volume triage, not detection, is the bottleneck.
Technical detail that earns credit	Name NCIIPC and CERT-In.
Suggested visual	One stat block.
What the presenter says	"More alerts do not help. Better decisions do."
Keep off this slide	No generic cybercrime statistics.

28.4 Slide 4: Existing System

Element	Content
Objective	Win the 25% criterion.
Points on the slide	Four-row grid: SIEM/UEBA (reports, does not forecast); LSTM and transformer classifiers (cannot condition on an action never taken); attack graphs (static reachability, no dynamics); KALACHAKRA (action-conditioned rollout).
Technical detail that earns credit	That a classifier <i>structurally</i> cannot represent an intervention - this is not a quality gap, it is a capability gap.
Suggested visual	A comparison grid, scannable in ten seconds.
What the presenter says	"There is no bolt-on that gives a classifier a counterfactual."
Keep off this slide	Never prose. A grid or nothing.

28.5 Slide 5: Proposed Solution

Element	Content
Objective	State the mechanism and show the evidence honestly.
Points on the slide	Graph state, GNN encoder, action-conditioned transition, K-step rollout, ATT&CK scoring. The 48/94/100 chart with the trivial baseline as the tallest bar. Stat cards.
Technical detail that earns credit	19.9M flows, 13 real captures, leave-one-scenario-out.
Suggested visual	Native bar chart; the trivial baseline highlighted in a warning colour.
What the presenter says	"The tallest bar is a trivial baseline that beats us, and we put it on the slide."
Keep off this slide	Do not put the refutations here - they belong on Feasibility.

28.6 Slide 6: Innovation

Element	Content
Objective	Say what is genuinely new.
Points on the slide	A capability class classifiers cannot have; latent prediction with collapse measured; an evaluation instrument; attribution chosen for a verifiable axiom.
Technical detail that earns credit	That the instrument caught an error in the team's own published result.
Suggested visual	Icon row.
What the presenter says	"Our own evaluation caught us under-reporting our own number, and we corrected it upward."
Keep off this slide	Never claim first or best.

28.7 Slide 7: Architecture

Element	Content
Objective	Prove technical depth in one image.
Points on the slide	The five-stage model diagram with the action input drawn into the transition model.
Technical detail that earns credit	JEPA-style latent objective, VICReg anti-collapse, GRU context, 7 ATT&CK stages, latent dimension 64.
Suggested visual	The architecture diagram (Figure 1 of this report).
What the presenter says	"The action input is the whole difference. A classifier has nowhere to put one."
Keep off this slide	Do not label every tensor shape.

28.8 Slide 8: Workflow

Element	Content
Objective	Show the counterfactual mechanism.
Points on the slide	One observed history, two rollouts, the gap between them.
Technical detail that earns credit	Risk aggregated as 1 minus the product of per-step survival.
Suggested visual	The branching diagram (Figure 2 of this report). This is the single strongest visual in the deck.
What the presenter says	"Same history, two actions, two futures. The gap is the model's belief about the intervention."
Keep off this slide	Do not clutter it with axis numbers.

28.9 Slide 9: Technologies

Element	Content
Objective	Answer the stack question.
Points on the slide	Python 3.11+, PyTorch, XGBoost, scikit-learn, NumPy, matplotlib, pytest. Console: plain HTML/JS, no framework, no CDN, no server.
Technical detail that earns credit	Why PyTorch: custom message passing, a custom rollout loop, and gradient access for integrated gradients.
Suggested visual	Compact stack table.
What the presenter says	"The console is a replay, not a live system, and we say so - a server would imply otherwise."
Keep off this slide	No transitive dependencies.

28.10 Slide 10: AI / ML

Element	Content
Objective	Show the modelling is deliberate.
Points on the slide	GNN encoder to a 64-dim latent; GRU context; action-conditioned transition; K-step rollout with predictions fed back; ATT&CK head on encoded and predicted latents; VICReg.
Technical detail that earns credit	The train/use mismatch that was found and fixed, and the collapse diagnostic (effective rank 24.65 to 41.11 of 64).
Suggested visual	The architecture diagram again, or a loss-term breakdown.
What the presenter says	"We report the collapse diagnostic because it is the known failure mode of this family and a judge will ask."
Keep off this slide	Do not show loss curves.

28.11 Slide 11: Features

Element	Content
Objective	Inventory concisely.
Points on the slide	Graph state; action-conditioned transition; K-step rollout; ATT&CK scoring; counterfactual rollout; integrated-gradients attribution; the lead-time harness.
Technical detail that earns credit	That attribution is <i>not</i> the SHAP or attention the PS names, and why.
Suggested visual	Two-column icon list.
What the presenter says	"We do not have SHAP or attention. We have integrated gradients, and we can verify its axiom on every call - which you cannot do cheaply with SHAP."
Keep off this slide	Do not overstate the counterfactual.

28.12 Slide 12: Demo Flow

Element	Content
Objective	Set expectations.
Points on the slide	The steps from Section 31.
Technical detail that earns credit	The console runs from the filesystem with no server.
Suggested visual	Three screenshots: case list, diverging trajectories, attribution panel.
What the presenter says	"This is a replay of a finished run. Nothing here is live and we do not pretend it is."
Keep off this slide	Do not narrate a demo you will also perform.

28.13 Slide 13: Results

Element	Content
Objective	The honesty slide. This is where the refutations live.
Points on the slide	Lead time: REFUTED on generated data and on real captures. Counterfactual: HOLDS, NARROWLY. Which interventions work: REFUTED after four attempts. Plus the self-correction from 1 of 12 to 5 of 12.
Technical detail that earns credit	5 of 12 against GBDT's 7 to 9; $r = -0.105, -0.103, +0.037, +0.011$; 0.03 to 0.99 AUC across captures.
Suggested visual	Three verdict cards with semantic colour - warning for REFUTED.
What the presenter says	"Our headline claim lost. We are showing you the curve."
Keep off this slide	Do not soften REFUTED to 'partially supported'.

28.14 Slide 14: Advantages

Element	Content
Objective	Translate into operator value.
Points on the slide	Answers a question class classifiers cannot; ATT&CK-native output; verifiable attribution; one core serving two problem statements.
Technical detail that earns credit	60-70% of the data foundation shared with EKAGRA.
Suggested visual	Four cards.
What the presenter says	"Even where the forecast loses, the evaluation is a contribution the field does not have."
Keep off this slide	Do not re-argue the refuted claim.

28.15 Slide 15: Future Scope

Element	Content
Objective	Show a prioritised, honest roadmap.
Points on the slide	Short: reframe around the instrument, pairwise-ranking metric, reconcile docs. Medium: a different formulation for the conditional effect, modern corpora, sparse graphs. Long: RL defender policy, a public benchmark.
Technical detail that earns credit	That four attempts at the conditional effect failed and the oracle says the limit is structural - so the fix is a new formulation, not another loss term.
Suggested visual	Three-horizon timeline.
What the presenter says	"We know the next thing to try, and we know why the last four things did not work."
Keep off this slide	Do not promise the RL policy as though it were nearly done.

28.16 Slide 16: Conclusion

Element	Content
Objective	One sentence.
Points on the slide	Built the world model the PS asks for; tested it against strong baselines on real data; published the loss; the evaluation caught an error in our own numbers.
Technical detail that earns credit	40 predictions, 16 refuted, verdicts printed by the runs.
Suggested visual	A single line.
What the presenter says	"Hand us a laptop and one command reproduces every number here, including the ones that went against us."
Keep off this slide	No new information.

29. Pitch Material

29.1 30-second pitch

"Most attack-prediction systems are classifiers - they score the traffic they saw. We built a world model instead: it learns how network state transitions, so it can roll forward and answer a question a classifier cannot even represent - what happens if we isolate this host right now. We tested it properly, including on thirteen real botnet captures, and our forecasting claim lost to a gradient-boosted tree. We publish that. What survives is the counterfactual, and an evaluation harness rigorous enough that it caught a mistake in our own published numbers."

29.2 1-minute pitch

Add: "The difference is structural, not incremental. A classifier learns $P(\text{label} \mid \text{what I saw})$. We learn $P(\text{next state} \mid \text{this state and an action})$. Conditioning on an action it only ever saw correlated with outcomes, a classifier learns the correlation - our action-aware LSTM baseline sits at chance. Ours does not. And we validate on matched pairs where the prefix is bit-identical before the fork, so any divergence is caused by the action rather than by noise. What we cannot yet do is tell you *which* of two hosts to isolate first - the model learns an average effect, and four separate attempts to recover a per-host effect all returned nulls."

29.3 3-minute pitch

Structure: (1) forecasting is a different task from detection, and why a classifier cannot represent an intervention; (2) the mechanism - graph state, latent transition with an action input, K-step rollout, ATT&CK scoring; (3) the evaluation - lead time at matched FPR, contained attacks as negatives, leave-one-scenario-out over thirteen real captures; (4) the result, led with honesty - 5 of 12 against gradient boosting's 7 to 9, and the lead-time claim refuted twice; (5) the correction story, which is the strongest thing in the project; (6) what survives - the counterfactual capability, the instrument, and a finding about the problem that applies to everyone.

29.4 5-minute technical explanation

Cover, in order: the graph state representation and why relational structure matters for lateral movement; the JEPA-style objective, the stop-gradient, and VICReg with the reported collapse diagnostic; the train/use mismatch that was found and fixed; multi-step rollout with prediction feedback and why start positions number T minus R ; compromise risk as one minus the product of survivals, and why that makes AUC incomparable across horizons; lead time at matched FPR with contained attacks as negatives; the three CTU-13 confounds and their fixes; nested selection; then the correction chain - deterministic control, six seeds, training length, and the AUC-versus-detection divergence.

29.5 Explaining it to a judge

Lead with the capability gap, not the accuracy number, because the accuracy number is not favourable. Then volunteer the refutation before they find it. Then pivot to the instrument: "the evaluation is rigorous enough that it caught us under-reporting our own result, and we corrected it upward against our own interest." That sequence turns the weakest part of the submission into the most credible part.

29.6 Explaining it to a non-technical audience

"Most security software is like a smoke alarm - it tells you there is a fire. We tried to build something more like a weather forecast: not just 'something is wrong' but 'here is where this is heading, and here is what changes if you act now'. We can show two futures side by side - one where you do nothing, one where you cut the machine off. Being honest: our forecast is not yet more accurate than much simpler methods, and we measured that carefully rather than claiming otherwise. What we do have that they do not is the ability to answer the 'what if we act' question at all."

29.7 Technical viva

Expect: why latent prediction rather than reconstruction; what VICReg does and what the effective rank means; why AUC is not comparable across horizons; what makes a negative a negative in the lead-time metric and why contained attacks are included; what nested selection prevents; why integrated gradients rather than SHAP and what completeness buys; how the deterministic control isolated the world-model path; and why the epochs finding invalidates the method but not the conclusion of experiment 15.

30. Expected Questions and Answers

Is this not just an LSTM with extra steps?

No, and the difference is testable rather than rhetorical. An LSTM learns $P(\text{label given the sequence it observed})$. We learn $P(\text{next state given this state and an action})$. We ran the fair version of your question: an action-aware LSTM, given the action as an input feature, sits at 48% - chance - on the counterfactual direction task, while the world model is at 82% in the same run. Given an action it only ever saw correlated with outcomes, a discriminative model learns the correlation, not the intervention.

How accurate is it?

On real botnet captures it catches 5 of 12 infections at a false-alarm rate at or below 10%. Gradient boosting catches 7 to 9. We lose. On generated data our window-level AUC is 0.784 against an LSTM's 0.968. We are not the most accurate model in this comparison and we do not claim to be.

Then why should we care about it at all?

Because accuracy on a detection task is not the only axis. The world model answers a question class the others cannot represent - what happens if we act. And the evaluation harness is a contribution in its own right: lead time at a matched false-alarm rate, contained attacks counted as negatives, leave-one-scenario-out over thirteen real captures. It is rigorous enough that it caught an error in our own published number.

Tell me about that error.

We published that the world model caught 1 of 12. Re-running our own configuration on our own folds gave 4 of 12. A deterministic control - gradient boosting, which has no training randomness here - reproduced its row to the digit, proving the pipeline was identical and isolating the difference to the world-model path. Six seeds ruled out noise. The cause was training length, and the direction is the interesting part: training longer raises AUC and lowers detection. Our tuning pass had selected on AUC, so it was steering away from the thing we actually report. The honest number is 5 of 12, and the error had made us look worse than we were. We fixed it anyway.

Your 94% counterfactual number - is that accuracy?

No, and this is important. It is the share of cases where the model says isolating helps. It never compares against an outcome, and isolation helps in every pair our generator makes - so a constant 'always isolate' predictor scores 100%. We put that 100% baseline on the slide next to our own number. What the figure does evidence is that an action-aware LSTM sits at 48%, which is chance. Note also that the strictly like-for-like comparison from a single run is 82% against 48%; the 94% is from a later variant trained with a paired-difference loss.

Why not SHAP or attention, as the problem statement asks?

Because we have neither and pretending otherwise would not survive one question. The encoder is normalised-adjacency message passing - there is no attention to read off. SHAP is a dependency plus thousands of forward passes per explanation. We ship integrated gradients, and the argument is one property: completeness, the guarantee that attributions sum to $f(\text{input})$ minus $f(\text{baseline})$. That is the same axiom SHAP's efficiency property gives, and unlike SHAP it is cheap enough to verify on every call - which we do, including in the browser.

What is your baseline for the attribution?

A quiet network, not zeros, and the reason is measured. An empty graph scores 0.768 compromise risk on the trained model; zeros with zero adjacency score 0.980 - the model finds 'nothing is happening' alarming. Against zeros we would be explaining only the sliver above 0.77, referenced to a state the network is never in. Against the benign mean of 0.027, attributions read as what makes this different from a normal day. Every result records which reference produced it.

Did you tune it properly, or just run defaults?

Both, and the second one is a better story. We ran a nested tuning pass - configuration chosen on an inner validation capture, never on the test fold - and it did not help. We then swept the rollout horizon, which that pass had deliberately

left alone. Even allowed to pick its horizon knowing the test results, which nothing deployable can do, the model reaches 5 of 12 against gradient boosting's 9. The upper bound loses, so no honest procedure could have rescued it. And along the way we found that our tuning pass had been selecting on the wrong metric.

What happens if the model fails?

It produces a forecast that is wrong, and on this evidence it will do that more often than a gradient-boosted tree would. There is no abstention mechanism in KALACHAKRA - that is EKAGRA's contribution. This is a research prototype, and it should not be the only thing an analyst relies on.

How does it scale?

Honestly, that is not established. The CTU-13 configuration uses 24 hosts and the message passing is dense. Training is around 15 seconds for 107 episodes at 12 epochs, so compute is not the constraint - data is. With twelve scored attack episodes, the variance between captures dwarfs the difference between configurations.

Why is cross-network transfer so poor?

It is poor for everyone, which is the finding. All four methods span roughly 0.03 to 0.99 AUC across the thirteen captures. At a standard deviation around 0.33, the four mean AUCs are not distinguishable from each other - ours included. Any team claiming a clean cross-network number on CTU-13 should be asked for their per-fold spread.

Why can you not test the counterfactual on real data?

Because no real capture contains the branch that did not happen. You cannot observe what would have followed had the host been isolated. So that claim stays on constructed matched pairs, where we control both branches, and we say so rather than reporting a real-data number that is not measuring it.

What are the edge cases and failure modes?

Three. The model predicts an average intervention effect, not a per-host one - four attempts to recover heterogeneity returned nulls at -0.105, -0.103, +0.037 and +0.011. Widening the rollout horizon makes our lead time worse, not better. And on CTU-13 the stage head degenerates to compromised-or-not, because that corpus has no kill-chain labels.

How would you deploy this at scale?

We would not, yet. There is no inference service, no streaming graph builder and no authenticated interface. What exists is a model, an evaluation and a replay console. Saying otherwise would be a claim we would have to walk back.

What would you improve with more time?

First, reframe the deliverable around the evaluation instrument, because that is the part that demonstrably works. Second, a pairwise-ranking counterfactual metric, where a constant predictor scores 0.5 by construction rather than 1.0 - though on our correlation evidence I expect that to return a null too, and it would be a better-posed null. Third, a genuinely different formulation for the conditional effect, because four loss-level attempts and an oracle diagnostic all say the limit is structural.

Why should we believe any of these numbers?

Because 16 of our 40 pre-registered predictions were refuted, the verdicts are printed by the runs themselves, and the one time our numbers disagreed with each other we chased it down with a deterministic control and published the correction. A result that only ever confirms its authors is the one to distrust.

31. Demonstration Script

Roughly three minutes. The console is a replay, so nothing can fail mid-demo for lack of a server.

#	Action	What should happen	Point to make
0	Before the room: confirm <code>kalachakra/frontend/data.js</code> exists. If not, run <code>python demo.py</code> - about seven minutes.	The payload is present.	Setup only.
1	Open <code>kalachakra/frontend/index.html</code> directly from the filesystem.	The console loads with no server running.	"No server, no CDN, no build step. This is a replay of a finished run, and we say so - a live server would imply something that is not true."
2	Select a case from the list.	The observed history for that episode is drawn up to its decision window.	"This is a held-out episode. The model has never seen it."
3	Show the two trajectories.	Two forward curves diverge from a shared prefix - 'do nothing' rising, 'isolate now' falling.	"Same history, two actions, two futures. That gap is the model's belief about the intervention - and a classifier cannot produce this picture at all."
4	Point at the realised outcome under the curves.	The ground truth for both branches is shown.	"These are matched pairs with bit-identical prefixes, so we know what actually happened in both branches. That is the only honest way to validate a counterfactual."
5	Open the attribution panel.	Ranked per-feature attributions appear with the reference state named and the completeness residual shown.	"The residual is verified in the browser before the console will call this ranking trustworthy. That is what completeness buys us and why we chose integrated gradients over SHAP."
6	If asked about accuracy: state the CTU-13 result before being pushed.	-	"On thirteen real botnet captures we catch 5 of 12 and a gradient-boosted tree catches 7 to 9. We lose, and the curve is in the repository."
7	Optional: <code>python experiments/exp18_ctu13_epochs.py</code> or show <code>results/exp18_verdicts.json</code> .	The epochs-versus-AUC divergence and all five verdicts.	"Training longer improves AUC and destroys detection. That is how we found the error in our own published number."

Table 44. Demonstration sequence.

32. Glossary

World model

A model that learns state-transition dynamics - how the world evolves - rather than a mapping from observations to labels. It can be rolled forward and conditioned on actions.

Transition dynamics $P(S_{t+1} | S_t, a)$

The probability of the next network state given the current state and an action taken. Learning this, rather than $P(\text{label} | \text{observation})$, is what makes the system a world model.

Counterfactual

A prediction about a scenario that did not occur - here, what would happen if a host were isolated when in fact it was not. Structurally unavailable to a discriminative classifier.

Host-interaction graph

The state representation: nodes are hosts, edges are observed communication in a window, with node, edge and global features.

GNN encoder / message passing

A neural network over graph structure in which each node's representation is updated from its neighbours. Here it is normalised-adjacency message passing, with no attention.

Latent state z

A 64-dimensional learned vector summarising the network's state in one window.

JEPA-style objective

Joint-Embedding Predictive Architecture: predict the *representation* of the next state rather than reconstructing raw input. Cheaper, and it does not waste capacity on noise.

Stop-gradient

Blocking gradient flow into the prediction target so the encoder cannot trivially satisfy the predictor by collapsing.

VICReg

Variance-Invariance-Covariance Regularisation. Penalises low variance and high covariance across latent dimensions to prevent collapse.

Latent collapse

The failure mode where the encoder maps every input to nearly the same vector, making prediction trivially perfect and the representation useless. Measured here via effective rank.

Effective rank

A continuous measure of how many latent dimensions are genuinely in use. Measured at 24.65 and 41.11 out of 64 - not collapsed.

K-step rollout

Applying the transition model K times, feeding its own predictions back in, to simulate forward without new observations.

Rollout depth / horizon

How many steps ahead the model is trained and asked to predict. These must match, which is why they are now passed together as arguments.

Compromise risk

P(reach LATERAL_MOVEMENT or beyond within the horizon), aggregated as 1 minus the product of per-step survival probabilities.

Lead time

Compromise window minus alarm window, clamped at zero. How much warning the defender actually got.

Lead time at matched false-alarm rate

The headline metric. Comparing raw lead times at different alarm rates is how a bad detector is made to look good.

Contained episode

An episode with real attack activity that never reaches compromise. Counted as a negative - which is what stops the task collapsing into detection.

Leave-one-scenario-out

Train on twelve CTU-13 captures, test on the thirteenth, rotated. Every test episode comes from a network and a malware family never seen in training.

Pseudo-replication

Treating correlated observations as independent - here, making one episode per infected host when all of them share pre-compromise context. It inflates n without adding evidence.

Nested selection

Choosing a configuration on an inner validation split that the test fold is not part of, so no reported number has seen its own test data.

Deterministic control

A component with no training randomness - here gradient boosting - used to prove that a pipeline is identical between runs, so any difference must lie elsewhere.

Integrated gradients

An attribution method integrating the gradient along a path from a reference state to the input.

Completeness

The axiom that attributions sum exactly to $f(\text{input})$ minus $f(\text{reference})$. The reason integrated gradients was chosen: it is verifiable on every call.

MITRE ATT&CK

A public taxonomy of adversary tactics. Used here as an ordered kill-chain ladder: BENIGN, RECON, INITIAL_ACCESS, EXECUTION, LATERAL_MOVEMENT, COLLECTION, EXFILTRATION.

CTU-13

Thirteen real botnet captures from Stratosphere Lab, CTU Prague (2011), totalling 19.9 million flows. The only real-attack corpus in this project.

Argus binetflow

The flow-record format CTU-13 ships in. Its State field encodes source flags before an underscore and destination flags after - only the source half is read here.

Pre-registered prediction

A hypothesis written into the experiment file before the first run, with the verdict printed by the run. Never edited afterwards.

33. One-Page Cheat Sheet

Project	KALACHAKRA - SIH26153 - NTRO - Team AlgoRhythms
Problem	A SOC needs to know how much time is left and whether acting helps. Detection scores answer neither, and a classifier structurally cannot condition on an action never taken.
Solution	A world model over a host-interaction graph: GNN encoder to a 64-dim latent, action-conditioned transition, K-step rollout, MITRE ATT&CK stage scoring, integrated-gradients attribution.
Target users	NTRO, NCIIPC-designated CII operators, CERT-In empanelled SOC analysts.
Core features	Graph state; action-conditioned rollout; counterfactual (two futures from one prefix); ATT&CK scoring; verified-completeness attribution; lead-time-at-matched-FPR evaluation harness.
Architecture	G _t -> GNN encoder -> (z, action) transition -> K-step rollout -> ATT&CK scoring. JEPA-style latent objective with VICReg.
Tech stack	Python 3.11+, PyTorch, XGBoost, scikit-learn, NumPy, matplotlib, pytest. Console: plain HTML/JS, no server, no CDN.
AI/ML	Latent predictive objective with stop-gradient; multi-step rollout with prediction feedback; VICReg anti-collapse (effective rank 24.65-41.11 of 64); integrated gradients with per-call completeness verification.
Key innovation	A capability class classifiers cannot represent - plus an evaluation instrument rigorous enough that it caught an error in the project's own published result.
Headline results	Lead time REFUTED on generated data (LSTM 3.35 vs 0.26 windows) and on CTU-13. Real data: 5 of 12 infections caught vs GBDT's 7-9. Counterfactual: 82% vs an action-aware LSTM's 48% in the same run (94% for a later variant) - but a trivial always-isolate scores 100%. 40 predictions, 16 refuted. 107 tests.
Advantages	Answers 'what if we act'; ATT&CK-native output; verifiable attribution; one core serves two problem statements; produces a finding about the problem (0.03-0.99 AUC across captures for every method).
Limitations	Forecasting loses to a gradient-boosted tree on real data. Predicts an average intervention effect, not a per-host one - four attempts failed. Widening the horizon makes our lead time worse. No deployment path. Counterfactual can never be validated on real captures.
Future scope	Short: reframe around the instrument, pairwise-ranking metric, reconcile docs. Medium: a different formulation for the conditional effect, modern corpora. Long: RL defender policy, a public benchmark.
One-line pitch	<i>"A model you can ask 'what if we act?' - and an evaluation honest enough to tell us our own forecast lost."</i>